OAuth App Setup Guides

An agent that tells a developer, for any app, how hard it will be to get their own OAuth app approved, how long it takes, what can block it, and which steps an agent can do for them.

Live: composio-oauth-guides-one.vercel.app

Code: github.com/manthanjha7/composio-oauth-guides

Case study online: manthanjha.vercel.app/case-studies/oauth-guides

Recorded live run: /generate?replay=asana

Built by: Manthan Jha, solo (research, product, build), October 2026

Live generation: on, capped at 10 guides a day

- **The problem is per setup, not per app.** Gmail is a 30-minute job for an internal tool and weeks of review plus a yearly security assessment for an agent that reads inboxes. Every guide rates each setup separately.
- **Two ways in:** a library of 11 guides, and a search box where the agent researches any app live in about 2 minutes and saves a shareable guide.
- **Every fact is checked in code:** its quote must appear word for word on the provider's own docs, with the same numbers. Library guides: 357 of 362 quotes verified.
- **A model of approval paths built from 86 providers:** 4 paths, 5 things a review checks, 7 factors. My first version fit 0 of 24.
- **Agent v2 tested on 18 apps chosen to cover every path:** 9 of 18 correct in the first round, 18 of 18 and 17 of 18 in the last two.

Review it in 5 minutes

1. **Open a guide:** on the live site, open Gmail and switch between setups. Difficulty, your time and Google's wait change with each.
2. **Check a fact:** open "Sources" on any line to see the exact quote and the page it came from. The "Verification" page shows how every quote was checked.
3. **Generate a new guide:** search an app not in the library, for example Trello or Linear. The agent researches it live in about two minutes. Live runs are capped at 10 a day; if the cap is reached, watch a recorded run.
4. **Read the code:** the repository. The agent is in `src/lib/agent/`, the evidence checks in `src/lib/evidence.ts`, every test round in `docs/agent/EVAL.md`.

1. The problem

A developer builds an AI agent on Composio. For a prototype, Composio lends its own approved OAuth apps and it just works. Launch is different: their own name on the consent screen, their own rate limits, extra permissions, or any of the 219 Composio toolkits with no shared app. Each means registering their own OAuth app with the provider and getting it approved.

That is where developers stall. Every provider runs approval differently, the rules are spread across many pages, and nobody says upfront whether it takes an hour or ten weeks.

Gmail, if you are building…	What Google asks for
An internal tool for your own company	Nothing. No review
A public app that only sends email	App verification, typically days
A public agent that reads inboxes on your servers	Verification, a third-party security assessment, and a re-check every year

Why it matters to Composio: Composio grows when agents make tool calls in production. A developer stuck in a provider's approval queue never gets there. This step sits between a free prototype and a real launch.

2. Learning the problem: two conversations

I had never registered an OAuth app that other people's accounts connect to. Before designing anything, I asked two engineers to walk me through it.

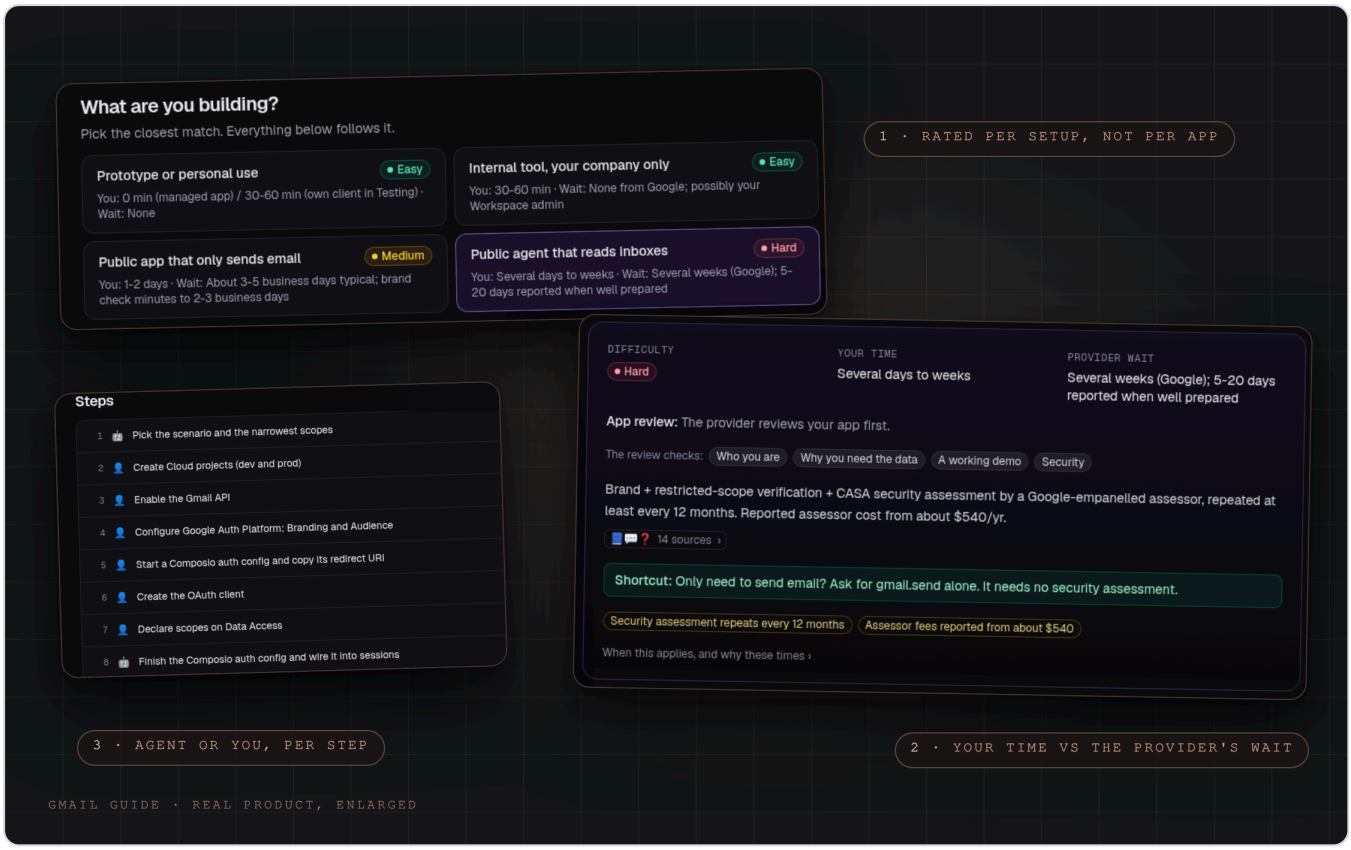
Who	What they told me	What it changed
Samir Backend engineer, pre-final year at IIT Roorkee	For every tool, the developer goes to that provider's developer site, creates an app, copies the client ID and secret, and pastes them into Composio. With several tools, they repeat this for each one, side by side, in a different console every time.	The work is repetitive and scattered, so an agent should do every step it can through an API and leave the developer only what needs a person: logging in, accepting terms, clicking approve. That became the **agent / you** label on every step.
Ayush Research intern at Adobe, final year at IIT Roorkee	The whole process end to end: what must exist before you start (a developer account, the app registration, a redirect address, the permissions you ask for), the order of the steps, and the points where it gets stuck.	Creating the app is quick. What decides whether launch takes an hour or weeks is what comes after: the review, and how long the provider takes. That led to rating each setup separately and showing **your time** apart from **the provider's wait**.

What I took from both conversations

- **The answer is needed before starting, not during.** A developer deciding which apps to support first needs "how hard and how long" upfront, so each guide opens with difficulty and wait, and the steps come after.
- **Small copy-paste details cause the failures.** A wrong redirect address or a missing permission only shows up at the end. Steps say exactly what to copy from where, including Composio's redirect address.
- **The same questions repeat for every tool.** So every guide uses the same format and the same research checklist, which is what makes a library of 100 apps comparable.

3. What I built

One search box. Type an app or a website: if a guide exists it opens instantly; if not, the agent researches it live, streams its progress, and saves a guide with a link anyone can open. Library cards show the easiest and the hardest way to set each app up, so "same app, different answer" is visible before you click.



The live Gmail guide, enlarged. The same app is Easy for an internal tool and Hard for an agent that reads inboxes; each setup splits your time from Google's wait and marks which steps an agent can do.

How a guide answers what the brief asked

The brief asked	Where the guide answers it
How easy or difficult it is	**"What are you building?"** comes first. Each setup gets Easy, Medium or Hard, and its approval path: self-serve, app review, or partner acceptance.
Time and effort, including the wait	**Your time** and **provider wait** are separate. "One hour of work plus six weeks of review" is a different plan from "a week of work". A wait with no source says "Unknown".
Requirements and nuances	**Have ready**, **Watch out**, and factors per setup: customer admin approval, limits before approval, user thresholds, paid access, use restrictions, re-reviews. Each one links to its source quote.
What an agent can do, where you take over	Every step is marked **agent**, **you** or **wait**. The agent configures through APIs; the human logs in, accepts terms, records the demo and clicks submit.
How to proceed when blocked	**Stuck?** lists the blockers developers hit (unverified-app screen, redirect-domain checks, rejected scopes) with what to do next.
Look up, share, generate live	Search, a share link and a Markdown copy for every guide, and live generation for apps outside the library, including apps Composio doesn't list.

4. How I approached it

1. **Mapped the journey first:** who is involved (developer, provider, Composio, the end user) and where the developer gets stuck. Flows and architecture on one page before any code.
2. **Had the plan reviewed twice**, by Codex and a separate Claude agent, independently.
3. **Checked my assumptions against real docs.** Three were wrong (below).
4. **Built, then verified every output against its sources**, and tested one setup end to end by hand.

The process, in order

#	Phase	What happened
1	Understand	Read the brief, Composio's docs and toolkit catalog; the two conversations above.
2	Plan	User flows and architecture on one page; reviewed independently by Codex and a Claude agent before any code.
3	First build	The site, the guide format, 4 researched guides, the first agent and the code checks on quotes. Slack set up end to end by hand.
4	Categorise	Tested approval categories against 86 providers over 7 rounds (section 5).
5	Redesign the guide page	Cut Gmail from 5,350 visible words to 750; the setup picker comes first; sources moved behind toggles.
6	Agent v2	Redesigned the agent around evidence, then had Codex review it end to end over 6 rounds until it judged the agent ready for live testing.
7	Test and fix	18 apps, 10 rounds, then checks on the real API (section 7).
8	Ship	Agent v2 live with a daily cap; code public; this write-up.

The decisions that shaped it

Decision	Options	What I chose, and why
Where knowledge comes from	A pre-built dataset vs live research every time	**Live research.** A dataset is blind to apps outside it and stale the day docs change. The library is saved agent output.
What gets a difficulty rating	One rating per app vs one per setup	**Per setup.** One rating per app is simple and wrong.
How time is shown	One number vs your time and the provider's wait	**Split.** The wait is the real risk and one number hides it.
How claims are trusted	Trust the model's citations vs check in code	**Check in code.** A link does not prove a claim. Anything that fails is shown as unconfirmed, not hidden.
Agent vs human	A bot that clicks through consoles vs APIs plus a human for logins and terms	**APIs plus a human.** Bots break on logins and two-factor. In the Slack test the agent did the setup and I clicked Allow once.

5. Categorising approval paths

To rate difficulty I needed categories. My first version was a ladder of five levels, from self-serve to security assessment. I tested it against 24 providers, each read from its own public docs by a cheap AI reader. **None fit cleanly.** So I looped: fold the misfits in, test 10 new providers, repeat. 86 providers over 7 rounds, until the categories stopped changing.

Round	Version	Clean fits
1	5-level ladder	0 of 24
2	8 independent gates	0 of 12
3	Two axes	0 of 10
4	+ decision rules	2 of 10
5	+ use restrictions	4 of 10
6–7	+ clarifications, stacked gates	3–4 of 10

The remaining misfits were mostly gaps in the providers' own docs, not gaps in the model.

The final model: three questions

1. **What must happen before an outside customer can connect?** Automatic, self-serve, app review, or partner acceptance. They can stack.
2. **If there is a review, what does it check?** Identity, use case and permissions, a working demo, security, listing content.
3. **What else changes the experience?** Customer admin approval, limits before approval, a user threshold, paid access, prerequisites, use restrictions, re-reviews.

What surprised me

- **It is not a ladder.** Steps stack in any combination.
- **"They review your app" and "they must accept your company" are different problems.** One is a checklist; the other can be a no.
- **101 of Composio's 343 OAuth toolkits need no app at all.** Their clients register themselves.
- **Customer-side setup is common.** At ServiceNow or Snowflake, each customer's admin creates the integration in their own account.

6. How the agent works

A fixed workflow, not a free-roaming agent: code decides the order and the budgets, models do the reading and writing. Every stage's model is configuration, so a run can use Opus for everything without code changes.

1. **Identify.** Composio's catalog plus a short search: which provider, its official website and docs. Ambiguous names get "did you mean".
2. **Anchor official sources.** Only the provider's own domains count as documentation. Look-alike domains and customer-hosted pages on shared platforms are excluded.
3. **Find pages.** Entry points, the docs' `llms.txt` index, and four fixed searches, ranked so approval, review and limits pages come first and API endpoint pages last.
4. **Read each page in full** against a research checklist of 13 questions (paths, review checks, limits, timelines, pricing, customer side, redirect rules). Each fact comes back with an exact quote.
5. **Fill gaps.** Up to 3 helpers search for whatever the docs didn't answer, plus developer reports for real wait times.
6. **Write.** The writer builds the guide only from collected evidence and cites evidence ids. Code fills in each quote and link, so the writer cannot invent a source.

7. **Check.** Code checks quotes and numbers. Then a judge model reads each claim, and each setup's path, wait and factors, against its quotes. Anything not supported is removed or marked unconfirmed.

Honest by design. If no source states a wait, the guide says "Unknown" instead of guessing. If the docs never mention a review for a setup, self-serve is shown as *inferred*, not stated.

7. How I tested it

The checker itself

I wrote 551 deliberately wrong versions of real claims (wrong numbers, swapped units, flipped "must" and "may"). Code alone rejects or flags 56% of them; the judge model then reviews every claim on the questions that matter. Of 349 correct claims, **none** are wrongly marked unconfirmed. 124 automated tests cover the pipeline.

A hands-on setup: Slack

The agent created the Slack app and the Composio auth config through their APIs. I clicked Allow once, the connection went live, and a real API call worked. It found three things the docs don't make clear: Composio's API rejects a field name used in its own docs example; its Slack toolkit asks for user scopes, not bot scopes; and connect links expire in 10 minutes.

An 18-app test covering every category

I picked 18 apps so that every path and factor appears at least once: API-key-only, self-serve, customer admin, app review (always, for sensitive permissions, above a usage threshold), partner acceptance, stacked gates, paid access, use restrictions. Each case lists the expected path, factors and must-have facts. Testing ran on my Claude subscription, so the API key was kept for production.

Round	All correct	What changed before it
1	9 of 18	First run of agent v2
2–4	12–14 of 18	Better page ranking; nested docs indexes; a pass that finds factors the writer left out
5–8	12–17 of 18	Shared doc platforms (developer.zendesk.com) treated as official; judge rules for "any of" statements
9–10	**18 and 17 of 18**	Judge moved from Haiku to Sonnet

Rounds 1 to 10 ran on my Claude subscription. Re-checks with the API's exact output limits passed 3 to 4 of 6 apps while Composio Search was returning empty results; every miss traced to pages that were never found, not to wrong facts.

What testing caught that reading the code didn't

- **The judge was the unstable part.** With Haiku as judge, the same agent scored 17 and then 12. Research was steady; the judge kept rejecting correct answers over wording. Sonnet held at 18 and 17 for about 3 cents more per guide.
- **Thinking can eat the answer.** On the API, the reading model spent its whole 3,000-token output limit thinking and returned no facts for Slack's most important pages. I raised the limit and made my test setup apply the same limits, so this can't hide again.
- **Wrong sources look right.** One run read six API endpoint pages for Gusto because their addresses contained "partner". In another, the writer's own note "no app review is mentioned" was read as evidence

that a review exists, which blocked a correct answer.
- **A bad page can mislead a good model.** The agent once said every Box enterprise admin must authorize your app. The page it read had dropped link text, so a rule for two app types read as a rule for all. Reading the docs' Markdown source fixed it.

Three of my own assumptions that were wrong

- **"Slack is fine without the Marketplace."** Unlisted apps get 1 request a minute when reading channel history.
- **"Shopify always needs review."** Custom apps for one store don't.
- **"Any restricted Gmail scope means a security assessment."** Only if the data reaches your servers.

---

8. Results

Number	What it means
11	guides: 4 researched ahead of time, 7 generated by the agent
357 of 362	library quotes found word for word on the provider's page
18 / 17 of 18	apps fully correct (path, factors, must-have facts) in the last two test rounds
$0.15–0.25	per generated guide on the API, about 2 minutes
0 of 349	correct claims wrongly marked unconfirmed by the checker

---

9. Limitations

- **No guide is human-reviewed yet.** Only Slack is tested hands-on; the badge on every guide says so.
- **The agent depends on search.** When Composio Search was slow during testing, discovery found fewer pages and guides got thinner. It now retries and reads in full any official pages its helpers find, but a search outage still lowers quality.
- **Review times are rarely published.** Guides say "Unknown" more often than a developer would like.
- **The agent can't see behind developer-console logins**, so anything only visible inside a console is out of reach.

---

10. What's next

- **Get to 100:** run the agent over Composio's 219 bring-your-own toolkits, and have a person review guides in order of how much each toolkit is used.
- **Show the guide where developers get stuck:** when they create an auth config, or when a user sees "this app is blocked".
- **Keep guides fresh:** re-check every quote weekly against the live page (the script exists) and re-run the agent when one breaks.
- **One metric:** time from a developer's first test connection to their first production connection on their own OAuth app.

---

11. What I learned

- **Setup was rarely the hard part.** Distribution rules, domain checks and the wait were.

- **"Unknown" is a valid answer.** A guide that guesses a wait time is worse than one that says it doesn't know.
- **Test your categories on real cases.** My first model fit 0 of 24 providers.
- **Spend on the step that decides what the user sees.** The cheapest model was fine for reading pages; the judge needed the stronger one.
- **Doing it finds what reading misses.** The Slack setup found three things the docs don't say.

Appendix A. How it is built

Part	What I used
Site	Next.js and TypeScript on Vercel; guides stored in Postgres (Neon)
Models	Claude Haiku 5.5 to identify the app and read pages; Claude Sonnet 5.5 to write the guide and to judge it. Each stage's model is a setting.
Composio	Its toolkit API for managed-app status and app details; Composio Search for finding pages
Checks	A fixed guide schema (Zod), quote and number checks in code, a judge model, 124 automated tests, 551 deliberately wrong claims to test the checker
Spend limits	A daily cap and a per-visitor hourly cap on live generation
How I worked	Built with Claude Code; plans and code reviewed by Codex; agent testing run on my Claude subscription so API credit was kept for production

Appendix B. Terms used

Term	Meaning
OAuth app	Your registered identity with a provider (for example Google): a client ID and secret, the permissions you request, and where users return after approving.
Client ID and secret	The OAuth app's credentials. You copy them from the provider's developer console into Composio.
Redirect address (redirect URI)	Where the provider sends a user after they approve. It must match exactly, and some providers verify that you own its domain.
Scopes	The permissions the app asks for. Sensitive ones usually trigger a stricter review.
Managed app vs your own app	Composio lends its own approved app for many tools, which is fine for a prototype. For launch, developers usually bring their own app, which means the provider's approval process.
Auth config, connected account	In Composio, the auth config holds your app's credentials; each end user who approves becomes a connected account.
App review vs partner acceptance	App review: the provider checks your app. Partner acceptance: the provider must accept your company first, and can say no.